



Security Review For Pred.app



Collaborative Audit Prepared For:
Lead Security Expert(s):

Pred.app
0x3b
thekmj

Date Audited:

November 3 - November 15, 2025

Introduction

PRED is a decentralised exchange for trading on sports outcomes. It has implemented a CLOB (Central Limit Order Book) structure in a trading terminal design, where traders with differing views are matched with each other. PRED features a high-speed order matching and settlement platform, which is implemented onchain.

Scope

Repository: `pred-org/neg-risk-ctf-adapter-private`

Audited Commit: `5eda75d0d7c2b9bd0e40425589e60b1280628f81`

Final Commit: `a2a79bf2ae80b61059a06bc133faf43af5664627`

Files:

- `src/CrossMatchingAdapter.sol`
- `src/libraries/CTHelpers.sol`
- `src/libraries/Helpers.sol`
- `src/libraries/NegRiskLib.sol`
- `src/modules/Auth.sol`
- `src/modules/MarketDataManager.sol`
- `src/NegRiskAdapter.sol`
- `src/NegRiskBatchRedeem.sol`
- `src/RevNegRiskAdapter.sol`
- `src/WrappedCollateral.sol`

Repository: `pred-org/uma-ctf-adapter-private`

Audited Commit: `7a5dc3ef192735cd2f6aa5dddf34fcabe9aba502`

Final Commit: `78fac351e963c080e6e74071ed3ac369b3790f3f`

Files:

- `src/libraries/AncillaryDataLib.sol`
- `src/TrustedCtfAdapter.sol`

Repository: `sherlock-scoping/pred-org__ctf-exchange-private`

Audited Commit: `b38bfdac1c6f1274b73f93a1e789baedb7d4e7d8`

Final Commit: `35332202ba7003fd80d5e37eca22031a91a51fef`

Files:

- src/exchange/BaseExchange.sol
- src/exchange/CTFExchange.sol
- src/exchange/libraries/CalculatorHelper.sol
- src/exchange/libraries/OrderStructs.sol
- src/exchange/libraries/TransferHelper.sol
- src/exchange/mixins/AssetOperations.sol
- src/exchange/mixins/Assets.sol
- src/exchange/mixins/Auth.sol
- src/exchange/mixins/Fees.sol
- src/exchange/mixins/Hashing.sol
- src/exchange/mixins/NonceManager.sol
- src/exchange/mixins/Pausable.sol
- src/exchange/mixins/PolyFactoryHelper.sol
- src/exchange/mixins/Registry.sol
- src/exchange/mixins/Signatures.sol
- src/exchange/mixins/Trading.sol

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
5	4	6

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue H-1: Vault does not grant approval to CrossMatchingAdapter. [RESOLVED]

Source: <https://github.com/sherlock-audit/2025-11-pred-app-nov-3rd/issues/29>

Summary

Vault does not grant approval to CrossMatchingAdapter.

Vulnerability Detail

Inside the vault, **no approval is granted to any contract.**

```
contract Vault is Auth, ERC1155TokenReceiver {
    using SafeTransferLib for ERC20;
```

This means that the following code, which attempts to borrow USDC, **will not be able to execute** because of the lack of a prior approval:

```
function _executeShortCrossMatch(
    Parsed[] memory parsedOrders,
    bytes32 marketId,
    uint256 totalSellUSDC,
    uint256 fillAmount
) internal {
    _collectBuyerUSDC(parsedOrders, true);

    // TotalBuyUSDC + TotalSellUSDC = (length of parsedOrders - 1) * fillAmount
    // Add the fillAmount to this amount to get the total USDC that we need to
    ↪ complete the split
    usdc.transferFrom(neg.vault(), address(this), fillAmount + totalSellUSDC);
```

Impact

The **whole contract will not function** as this part reverts due to the lack of approval.

Tool Used

Manual Review

Recommendation

Make sure the vault gives an approval to the adapter.

Discussion

midori-fuse

Note for fix: Vault has functions for authorized addresses to pull funds

<https://github.com/sherlock-audit/2025-11-pred-app-nov-3rd/blob/main/neg-risk-ctf-adapter-private/src/Vault.sol>
[allowbreak #L26-L28](#)

vaibhav-pred

We give this approval manually using deployment scripts

midori-fuse

In the first version of the audit scope, the approval was given using `vm.prank()` directly on a contract. This is not possible on mainnet, and the issue itself was also fixed mid-audit.

midori-fuse

Fix confirmed. Direct WCOL minting is now used, no longer requiring funds from the vault.

Issue H-2: Converting positions on the RevNegRiskAdapter will not work because it has no minting rights over WCOL [RESOLVED]

Source: <https://github.com/sherlock-audit/2025-11-pred-app-nov-3rd/issues/33>

Summary

Converting positions on the RevNegRiskAdapter will not work because it is reliant on WCOL minting, but has no rights of it.

Vulnerability Detail

RevNegRiskAdapter.convertPositions() is reliant on wcol.mint():

```
// **Seed:** adapter mints **A WCOL** once.  
wcol.mint(_amount);
```

Minting WCOL requires ownership rights:

```
/// @notice Mints the specified amount of tokens  
/// @notice Can only be called by the owner  
/// @param _amount - the amount of tokens to mint  
function mint(uint256 _amount) external onlyOwner {  
    _mint(msg.sender, _amount);  
}
```

<https://github.com/sherlock-audit/2025-11-pred-app-nov-3rd/blob/main/neg-risk-ctf-adapter-private/src/WrappedCollateral.sol#L82-L87>

The problem is, WrappedCollateral is deployed by the NegRiskAdapter at its own deployment:

```
constructor(address _ctf, address _collateral, address _vault) {  
    // ...  
    wcol = new WrappedCollateral(_collateral, col.decimals());  
    // ...  
}
```

<https://github.com/sherlock-audit/2025-11-pred-app-nov-3rd/blob/main/neg-risk-ctf-adapter-private/src/NegRiskAdapter.sol#L68>

and that makes the Negrisk adapter to be the wcol's sole owner. There's no way for anyone else to add owners into the Wrapped Collateral, so wcol non-public functions would be impossible to call.

Impact

Reverse Negrisk Adapter's main functionality will not work. This in turn bricks the Cross-Matching Adapter as well.

Code Snippet

Tool Used

Manual Review

Recommendation

Allow a controlled address to gain some rights to the WCOL, and add ownership rights to the Reverse Negrisk Adapter.

Discussion

vaibhav-pred

We give this access manually using deployment scripts

midori-fuse

In the first version of the audit scope, the access was given using `vm.prank()` directly on a contract. It's not possible to "prank" a contract on mainnet.

Furthermore, in addition to having no recovery method using purely deployment scripts (considering the old scope), it was also fixed in the new scope of the audit, hence we have to consider the issue to have been discovered.

midori-fuse

Fix confirmed. WCOL now allows adding new admins, which will enable a full setup where all contracts have minting rights.

An additional `renounceOwnerRole` is added, enables removal of any unnecessary access points (e.g. deployer wallets) after the protocol is fully set up.

Issue H-3: YES tokens can be minted for free by circling the two Negrisk conversions during question deployment [RESOLVED]

Source: <https://github.com/sherlock-audit/2025-11-pred-app-nov-3rd/issues/34>

Summary

The process of preparing a question on the Negrisk adapter can be sandwiched by an adversary, minting outcome tokens without costing any collateral.

Vulnerability Detail

The flow for the admin to prepare a negrisk market is:

1. Call `prepareMarket()` on the Negrisk operator contract - will in turn call that on the negrisk adapter
2. Call `prepareQuestion()` on the Negrisk operator contract, which will add a question into the corresponding negrisk market - will in turn call that on the negrisk adapter

For example, if a negrisk market has 4 YES/NO questions (where exactly one resolves to YES), then you need one prepare market, and four prepare question calls.

Adversary can mint YES for free during the market setup by sandwiching the admin preparing a question. A detailed attack is as follow.

PoC

Let's say a market has 3 teams already, A, B, C. Denote the YES X by YX defining "team X wins the game", and NO X by NX. Let's say there's a fourth team, D, joins the tournament, or simply the admin has already set up the question for team A+B+C in the market, and is in the process of setting up the question for team D. We can mint a YES D for free this way:

1. Front-run step 2: Mint a set of YA NA and YB NB, let's say 1 token each. Use the reverse negrisk to convert the YA and YB into 1 token NC
 - Our balance is now -2 USDC, 1 NA, 1 NB, 1 NC
2. Admin creates the D question in the market.
3. Back-run step 2: Convert the NC into YA, YB, YD using the normal negrisk.
 - Our balance is now -2 USDC, 1 NA, 1 NB, 1 YA, 1 YB, 1 YD
4. Merge the NA, YA and NB, YB back into 2 USDC. We are left with only the 1 YD

We have just minted a YES D without collateral. All it takes now is to obtain a NO D, for example by buying on the market when the admin opens it for trading, and combine with that YES D to get 1 USDC profit.

Impact

YES tokens can be minted without any collateral on Negrisk markets. This allows undercollateralized betting, and WCOL funds can be stolen.

Code Snippet

<https://github.com/sherlock-audit/2025-11-pred-app-nov-3rd/blob/main/neg-risk-ctf-adapter-private/src/NegRiskAdapter.sol#L389-L405>

Tool Used

Manual Review

Recommendation

Full fix is non-trivial, but an easy fix would be to disable both adapters during market setup (for the market being set up only).

- There should be no "teams joining the tournament" once the market is fully setup. If adding new teams/questions is desirable, then this fix won't work, and a different fix is needed.

Discussion

vaibhav-pred

We have added a setPrepared function in src/NegRiskAdapter.sol Rev Neg Risk Adapter is only activated post setPrepared

midori-fuse

Fix confirmed. The reverse negrisk adapter can now only be used when the market is marked as fully prepared.

The same restriction does not apply to the negrisk adapter, however the attack is not possible with only negrisk conversion available. Negrisk conversion during market preparation is strictly detrimental and footgun, as converting too early only results in less NO minted.

Issue H-4: CMA does not transfer the fees correctly [RESOLVED]

Source: <https://github.com/sherlock-audit/2025-11-pred-app-nov-3rd/issues/44>

Summary

CMA matches orders, but doesn't collect the fees from them, which leads to the tokens being stuck inside the contract.

Vulnerability Detail

When matching order we charge a fee and send it to the `msg.sender`, which in most cases is the system controlled address that matches orders.

```
function _matchOrders(
    OrderIntent memory takerOrder,
    OrderIntent[] memory makerOrders,
    uint256 takerFillAmount,
    uint256[] memory makerFillAmounts
) internal {
    //...

    // Send the fee to msg.sender, CMA in our case
    _chargeFee(address(this), msg.sender, takerAssetId, fee);
}
```

However the CMA also has the ability to match orders, where the fee would be sent to it.

```
function hybridMatchOrders(
    // ...
    if (singleOrderCount > 0) {
        ctfExchange.matchOrders(takerOrder, singleMakerOrders,
            ↪ singleOrdersTakerAmount, singleMakerFillAmounts);
    }
}
```

However CMA lacks a way to get the fees out of the contract, which will lead to them being stuck.

Impact

Stuck fees inside the CMA.

Tool Used

Manual Review

Recommendation

Add a mechanism to record the fees and make sure to collect them after we call `ctfExchange.matchOrders`.

Issue H-5: Orders matched at above 100% cause the difference to get stuck inside the contract [RE-SOLVED]

Source: <https://github.com/sherlock-audit/2025-11-pred-app-nov-3rd/issues/45>

Summary

Orders matched above 100% cause the difference to get stuck inside the contract.

Vulnerability Detail

When buying tokens we require the total combined price to be $\geq 1e6$ (which is correct), but we miss to handle the case where the price combined price is $\geq 1e6$.

```
function _parseMakerOrdersLong(
    //...
    // Validate that the combined price of all orders equals or exceeds 1
    if (totalCombinedPrice < ONE) {
        revert InvalidCombinedPrice();
    }
}
```

User can set order, but the market may execute them later at a different price, and since we don't allow the price to be insolvent (which is correct) we always aim for at least 1 or more. This will cause some orders to be matched at a positive and will collect more funds from buyers than the contract will send to the sellers, which will result in tokens being stuck.

Example:

1. Price is 0.2, 0.3, 0.25, 0.25
2. User creates an order to buy 100 YES_B - 0.30 → pays 30 USDC
3. Price changes to: 0.25, 0.25, 0.25, 0.25
4. User is fulfilled and pays extra for his tokens

Final result is Taker: SELL 100 NO_A - 0.25 → wants 75 USDC (market order) Maker1: BUY 100 YES_B - 0.30 → pays 30 USDC (limit order from step 2) Maker2: BUY 100 YES_C - 0.25 → pays 25 USDC (market order) Maker3: BUY 100 YES_D - 0.25 → pays 25 USDC (market order)

Total price: $0.25 + 0.30 + 0.25 + 0.25 = 1.05$

Impact

Users buy orders at unfair prices and their tokens get stuck inside the contract.

Tool Used

Manual Review

Recommendation

Consider redistributing the extra to the users as lower prices on buys or higher prices on sells.

Issue M-1: Taker orders are always parsed as if they are maker [RESOLVED]

Source: <https://github.com/sherlock-audit/2025-11-pred-app-nov-3rd/issues/30>

Summary

Orders are always parsed as if they are maker orders.

Vulnerability Detail

Inside `_parseOrder`, the fee is always calculated using `ctfExchange.FEE_RATIO()`, which represents the maker fee ratio:

```
uint256 feeAmount = CalculatorHelper.calculateFee(order.order.feeRateBps,  
↪ order.side == Side.BUY ? takingAmount : makingAmount, order.makerAmount,  
↪ order.takerAmount, order.side, ctfExchange.FEE_RATIO());
```

However, no matter what order is being processed, it is always parsed as such, even the taker orders inside `crossMatchShortOrders`:

```
parsedOrders = new Parsed[] (multiOrderMaker.length + 1);  
parsedOrders[0] = _parseOrder(takerOrder, fillAmount, takerFillAmount,  
↪ takingAmount);
```

This means that the fees for the taker are being lowered.

Impact

Takers and Makers inside the CMA have the exact same fee rates.

Tool Used

Manual Review

Recommendation

Consider changing the function to accept a variable to determine if the user is a taker or a maker and then switch to the default fee rate if the user is a taker.

Example code:

```
function _parseOrder(  
    OrderIntent calldata order,
```

```

uint256 fillAmount,
uint256 makingAmount,
uint256 takingAmount
+   bool isMaker
) internal view returns (Parsed memory) {

    uint256 feeAmount = CalculatorHelper.calculateFee(
        order.order.feeRateBps,
        order.side == Side.BUY ? takingAmount : makingAmount,
        order.makerAmount,
        order.takerAmount,
-       order.side,ctfExchange.FEE_RATIO()
+       isMaker ? ctfExchange.FEE_RATIO() : ctfExchange.BPS_DIVISOR()
    );

```

Discussion

midori-fuse

Fix confirmed. Matched maker and taker orders now use the correct fee rate.

Issue M-2: Hardcoded DELAY_PERIOD on the Negrisk Operator forces an unnecessary 1 hour wait on all market resolutions [RESOLVED]

Source: <https://github.com/sherlock-audit/2025-11-pred-app-nov-3rd/issues/35>

Summary

DELAY_PERIOD is hardcoded and not overridden on the NegriskOperator. This forces an unnecessary delay on the resolution and reward claiming of all markets.

Vulnerability Detail

The flow for question resolution on a negrisk market is that:

- The oracle adapter (in our case, trusted CTF adapter) calls `reportPayouts()`
- DELAY_PERIOD later, anyone can call `NegRiskOperator.resolveQuestion()`. This is to give time for people to dispute, or for the admin to flag suspicious outcomes.
- After this, the negrisk markets can be paid out.

In the classic UMA oracle setup, the 1-hour delay is there for emergency admin flags and disputes. However, in our case, there's not supposed to be any delay because outcomes come from the trusted oracle. Hence this delay period forces an unnecessary 1-hour wait onto the resolution of all negrisk markets.

Impact

All market resolutions are delayed unnecessarily by 1 hour.

Multi-team fast markets, for example "who wins this F1 race", or similar Olympic events such as running or swimming, where many events are held in quick succession, are more impacted due to their fast-paced nature.

Code Snippet

<https://github.com/sherlock-audit/2025-11-pred-app-nov-3rd/blob/main/neg-risk-ctf-adapter-private/src/NegRiskOperator.sol#L50>

<https://github.com/sherlock-audit/2025-11-pred-app-nov-3rd/blob/main/neg-risk-ctf-adapter-private/src/NegRiskOperator.sol#L183-L185>

Tool Used

Manual Review

Recommendation

Override the `DELAY_PERIOD` to zero, or remove it entirely

Discussion

vaibhav-pred

We have removed this from `src/NegRiskOperator.sol`

midori-fuse

Fix confirmed. `DELAY_PERIOD` is no longer used on `NegRiskOperator`, allowing immediate resolution of any markets as soon as the market result is determined and verified by the admin.

Issue M-3: `_validateIntentConstraints` on the Buy side can be force-failed by sending counter-tokens directly to the maker [RESOLVED]

Source: <https://github.com/sherlock-audit/2025-11-pred-app-nov-3rd/issues/37>

Summary

`_validateIntentConstraints` can be weaponized to deny all Buy orders by sending the counterpart token to the order maker

Vulnerability Detail

`_validateIntentConstraints()` validates that, if the side of the order is Buy, then the user doesn't hold the counterpart token of their intent. For example, if the order is a Short + Buy (i.e. Buy No), then the user shouldn't be holding Yes tokens. The intention is to prevent a user from holding two options of the same question.

The problem is that, this check can be force-failed by *other people* sending a tiny amount of the counter-token to the order maker. The amount can be as small as 1 wei, and the order would still fail.

The only way for the operator to resolve it is to match the 1-wei sell order, which incurs an operational cost, and the scenario is repeatable.

Impact

Buy side orders can be force-failed

Code Snippet

https://github.com/sherlock-audit/2025-11-pred-app-nov-3rd/blob/main/pred-org__ctf-exchange-private/src/exchange/mixins/Trading.sol#L287-L301

Tool Used

Manual Review

Recommendation

We suggest not checking this on the onchain level, but rather on the off-chain level.

For example if a user already holds NO tokens, but wants to Long, then split their Long order into two separate fills, a sell and then a buy, based on their balance at the time of checking. The sell can be potentially skipped offchain if the volume is too small.

Discussion

midori-fuse

Fix confirmed. `_validateIntentConstraints()` is no longer used, thereby no longer blocking any orders based on users' balance.

Issue M-4: Cross Matching Adapter has no access control [RESOLVED]

Source: <https://github.com/sherlock-audit/2025-11-pred-app-nov-3rd/issues/38>

Summary

The Cross Matching Adapter, does not have an access control - anyone can match signed orders. This is problematic, and we present several impacts arising from this.

Vulnerability Detail

The CTF token exchange has an access control in order matchings - only authorized operators can match orders.

```
function fillOrder(OrderIntent memory order, uint256 fillAmount) external  
    ↪ nonReentrant onlyOperator notPaused {  
        _fillOrder(order, fillAmount, msg.sender);  
    }
```

The Cross Matching Adapter on the other hand, does not. Anyone can match orders.

This allows for several impacts:

- Full-filling orders on the CTF can be denied by filling the exact same order with 1 wei on the Cross Matching Adapter.
- Because maker and taker fee rates are different, anyone can fill their own orders with others' compatible orders, except that they are able to choose the cheaper fee side for themselves.

Impact

See above

Code Snippet

<https://github.com/sherlock-audit/2025-11-pred-app-nov-3rd/blob/main/neg-risk-ctf-adapter-private/src/CrossMatchingAdapter.sol#L153-L159>

<https://github.com/sherlock-audit/2025-11-pred-app-nov-3rd/blob/main/neg-risk-ctf-adapter-private/src/CrossMatchingAdapter.sol#L221-L227>

<https://github.com/sherlock-audit/2025-11-pred-app-nov-3rd/blob/main/neg-risk-ctf-adapter-private/src/CrossMatchingAdapter.sol#L382-L388>

Tool Used

Manual Review

Recommendation

Only allow authorized matchers to match orders on the Cross Matching Adapter

Discussion

midori-fuse

Fix confirmed. All order matching entrypoints now have access control

Issue L-1: Some variables are not used anywhere [RE-SOLVED]

Source: <https://github.com/sherlock-audit/2025-11-pred-app-nov-3rd/issues/31>

Summary

Some variables are not used anywhere

Vulnerability Detail

`error TransferFailed()`; is not used anywhere inside `NegRiskBatchRedeem`, it can be removed.

Tool Used

Manual Review

Recommendation

Consider removing it.

Discussion

midori-fuse

Fix confirmed. The unused custom error is no longer present

Issue L-2: Wrong event emission [RESOLVED]

Source: <https://github.com/sherlock-audit/2025-11-pred-app-nov-3rd/issues/32>

Summary

Wrong event emission

Vulnerability Detail

Inside NegRiskBatchRedeem we have BatchRedemption, where we would focus on uint256[] indexed amounts, which should be the user payments.

```
event BatchRedemption( bytes32 indexed questionId, address[] indexed users,  
    ↳ uint256[] indexed amounts, uint256 totalPayout );
```

However that event emits _yesAmounts instead.

```
for (uint256 i = 0; i < _users.length; i++) {  
    address user = _users[i];  
    uint256 yesAmount = _yesAmounts[i];  
    uint256 noAmount = _noAmounts[i];  
  
    // Redeem positions for this user with custom amounts  
    uint256 userPayout = _redeemUserPositions(conditionId, user, yesPositionId,  
        ↳ noPositionId, yesAmount, noAmount);  
    totalPayout += userPayout;  
}  
  
emit BatchRedemption(_questionId, _users, _yesAmounts, totalPayout);
```

Tool Used

Manual Review

Recommendation

Consider changing that variable to an array of userPayout for more accurate event emission.

Discussion

midori-fuse

Fix confirmed. `BatchRedemption` event now includes redemption info on both YES and NO amounts. Any instance of event emission uses the correct information.

Issue L-3: questions[questionID] is never initialized [RESOLVED]

Source: <https://github.com/sherlock-audit/2025-11-pred-app-nov-3rd/issues/36>

When the admin initializes a question, through `TrustedCtfAdapter.initialize()`, the mapping `questions` is never set.

This does not enable re-initializing a question, because we have the `require(!m.prepared, "prepared");` below. However questions mapping would never store any actual data, and so we have no such information stored persistent onchain.

Discussion

midori-fuse

Fix confirmed. Question data is now saved onchain.

Issue L-4: Pulling YES tokens in during RevNegRiskAdapter.convertPositions() can be done with batch transfer for gas efficiency [RESOLVED]

Source: <https://github.com/sherlock-audit/2025-11-pred-app-nov-3rd/issues/39>

The Reverse Negrisk Adapter allows conversion of N – 1 YES positions into the other NO position. For each YES token, it pulls from the `msg.sender` address to the burn address

```
ctf.safeTransferFrom(msg.sender, YES_TOKEN_BURN_ADDRESS, yesPositionId, _amount,  
    ↪ "");
```

Because the source and destination of the transfer are the same, this can be changed into a batch transfer, which will save many external calls and saves gas.

Discussion

midori-fuse

Fix confirmed. Batch transfer is now used rather than individual transfers.

Issue L-5: RevNegRiskAdapter.mergeAllYesTokens() pull in the wrong amount of pivot tokens [RESOLVED]

Source: <https://github.com/sherlock-audit/2025-11-pred-app-nov-3rd/issues/41>

Vulnerability Detail

`mergeAllYesTokens()` allows the user to merge a full set of YES tokens into 1 USDC. It does so by pulling in all except a pivot YES tokens in a set into the pivot NO, then pull in the pivot NO to merge into 1 USDC.

The conversion of $N - 1$ YES into a NO is done using `convertPositions`, which takes a fee on the proceedings. However, when pulling in the pivot YES:

```
// Transfer the YES tokens from the user to the adapter for merging
uint256 yesPositionId = neg.getPositionId(NegRiskIdLib.getQuestionId(_marketId,
↪ uint8(_pivotId)), true);
ctf.safeTransferFrom(msg.sender, address(this), yesPositionId, _amount, ""); //
↪ @audit uses _amount instead of actualNoAmount
```

the amount of pivot YES pulled in is `_amount`, rather than the amount of NO proceedings (which is `actualNoAmount`). i.e. the pivot YES amount pulled in is the pre-free NO, instead of post-fee NO.

This leaves some YES tokens stranded in the Adapter (equal to the NO fee collected)

Recommendation

Pull in `actualNoAmount` rather than the `_amount`, or also send the remaining YES amount as fee

Discussion

vaibhav-pred

We will never enable fees in Convert Positions

midori-fuse

Acknowledged. Fees will not be enabled for conversion operations.

Issue L-6: Cross Matching Adapter never takes into account the Negrisk/Reverse Negrisk conversion fee when matching [RESOLVED]

Source: <https://github.com/sherlock-audit/2025-11-pred-app-nov-3rd/issues/42>

Vulnerability Detail

The Cross Matching Adapter allows for matching orders of different markets belonging to the same Negrisk group. It does so by utilizing cross-question conversion on the Negrisk and Reverse Negrisk adapters.

The problem is that, converting questions on the Negrisk/RevNegrisk adapters take a fee, however the CMA works on the assumption that the input amount is the same as the output amount, not taking into account conversion fee.

Recommendation

Take into account conversion fee

Discussion

vaibhav-pred

We will never enable fees in Convert Positions

midori-fuse

Acknowledged. Fees will not be enabled for conversion operations.

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.